Відокремлений структурний підрозділ
«Економіко-технологічний фаховий коледж
Херсонського національного технічного університету»

ПРАКТИЧНА РОБОТА № 3

з дисципліни «Проєктний практикум»

Тема: «Проєктування архітектури даних та інтерфейсу»

Проєкт «EasyStock»

Виконав:
студент групи ЗІПЗс-24
Бондаренко Сергій

Перевірив:
Ігор Косенко

Хмельницький 2026

Мета вимоги та контекст системи

Я самостійно спроєктував архітектуру, модель даних, прототип і два ADR для EasyStock. Головне рішення — модульний Flask застосунок з React інтерфейсом і однією транзакцією для зміни залишку та аудиту.

Вимоги що визначають проєктування	Архітектурний наслідок
US-01 вхід; NFR-03 доступ	Auth і серверні сеанси; контроль ролей на API
US-03 пошук; NFR-01 швидкодія	Пошукове поле casefold, пагінація, індекси
US-08/09 операції; NFR-02 цілісність	Stock Service, умовний UPDATE, транзакція
US-11 історія; NFR-06 аудит	Незмінні через API записи StockMovement



Ролі користувачів продукту. Єдиний розробник — Бондаренко Сергій.

Рисунок 1. Контекст EasyStock як єдиної системи.

Постачальник є записом довідника, а не підключеним зовнішнім сервісом. SQLite і API належать внутрішній реалізації й не показані як зовнішні системи на контекстній схемі.

Компоненти та розгортання

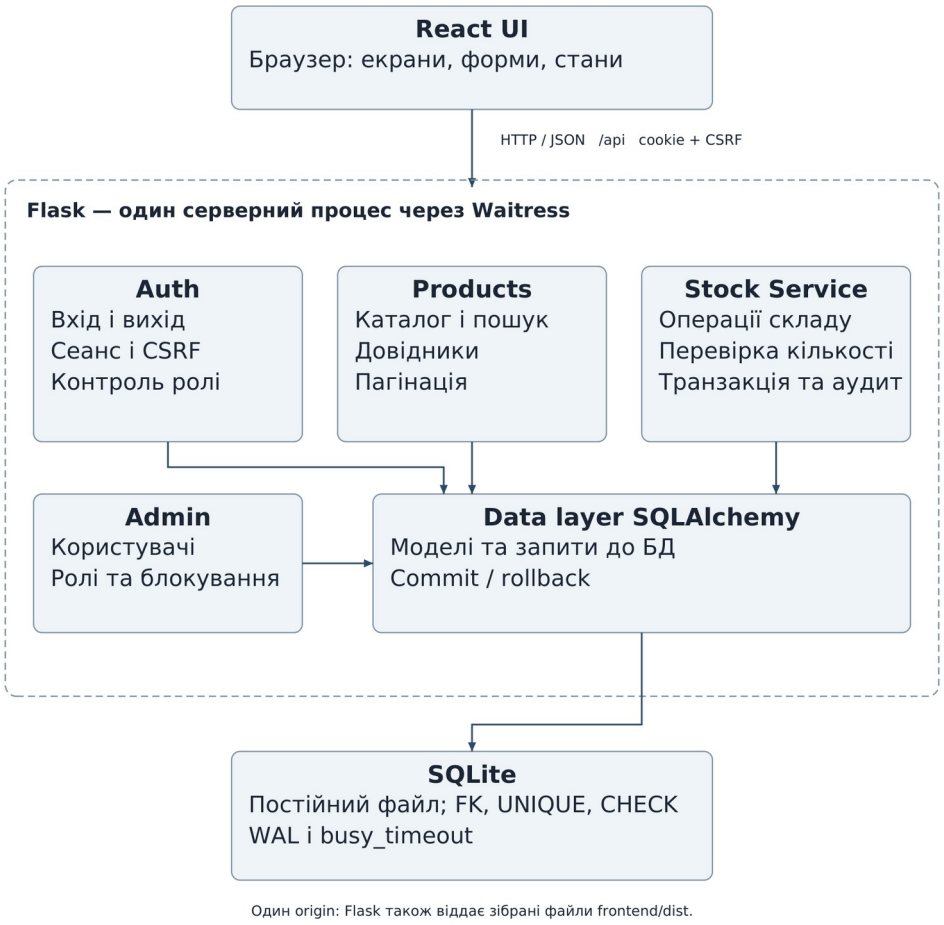


Рисунок 2. Сім логічних компонентів і локальне розгортання.

Компонент	Відповідальність
React UI	Навігація, форми, введення та стани відповіді
Auth	Сеанс, CSRF і перевірка ролі
Products	Каталог, пошук, категорії та постачальники
Stock Service	Валідація операції, баланс, документ і аудит
Admin	Користувачі, роль, пароль і блокування
Data layer	SQLAlchemy моделі, запити та транзакції
SQLite	Збереження, FK, UNIQUE, CHECK, WAL

Це логічні компоненти модульного моноліту, а не окремі мережеві сервіси. У production збірці Waitress запускає Flask, який віддає API і frontend/dist з одного origin. У розробці Vite проксіює /api до Flask.

Критичний сценарій списання

Передумова: комірник авторизований; SSD-480 після надходження має 37 одиниць. Основний потік списує 3 до залишку 34. Альтернативний потік намагається списати 40 і зберігає дані без зміни.

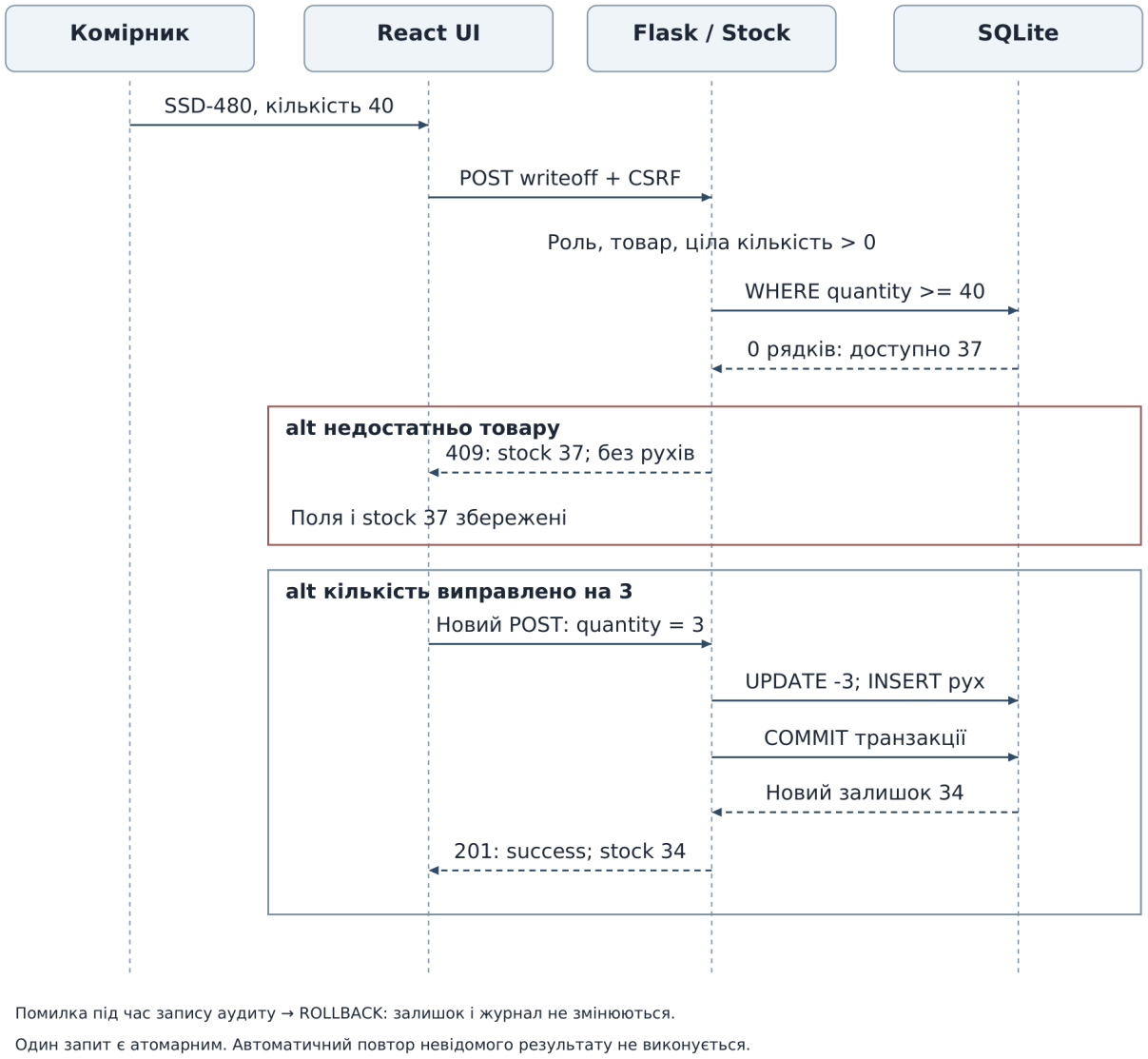


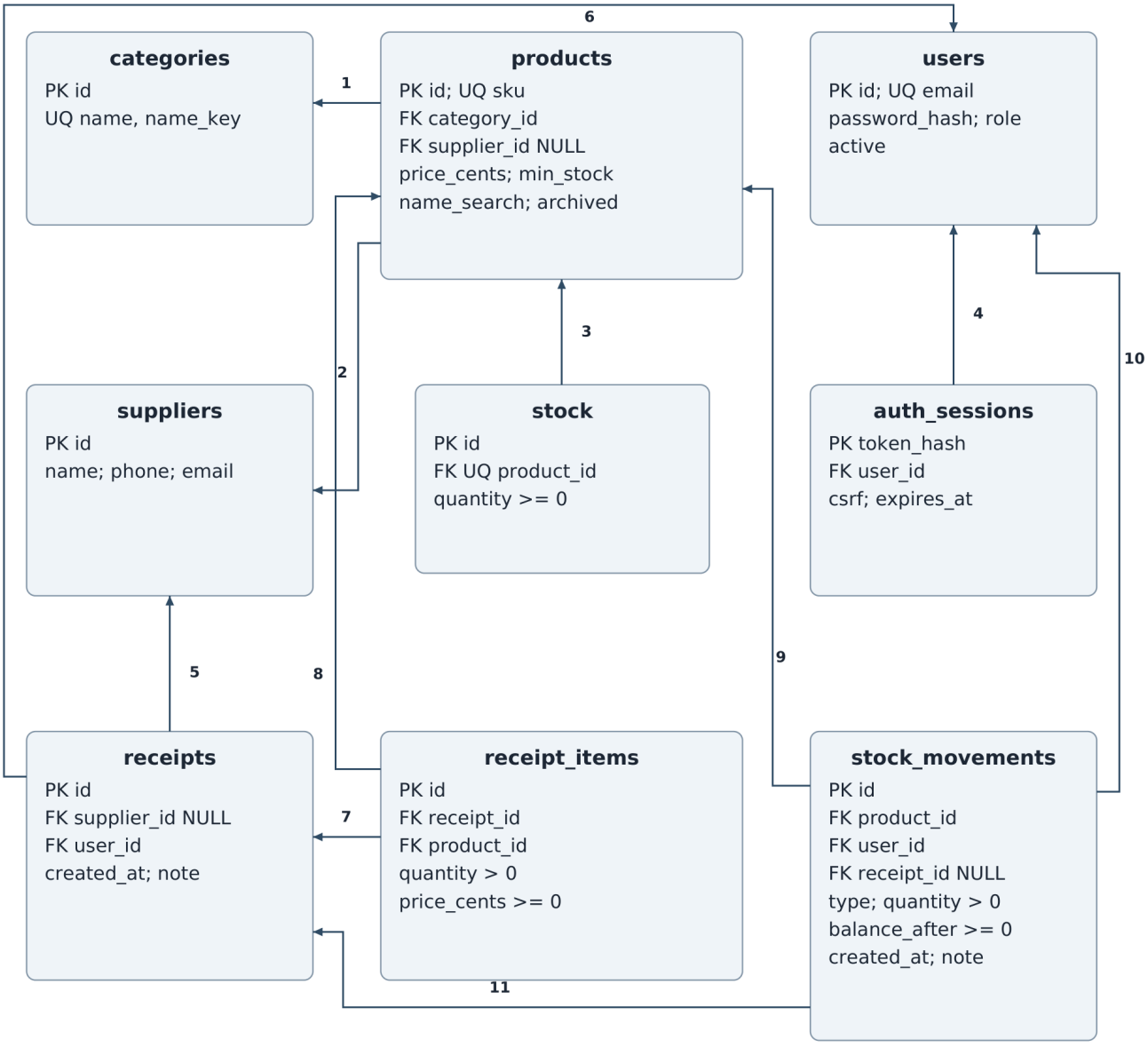
Рисунок 3. Списання з відхиленням надмірної кількості і успішним виправленням.

API перевіряє роль, CSRF, активний товар і цілу додатну кількість. Умовний SQL decrement змінює залишок лише за достатньої наявності. Рух записується у тій самій транзакції; помилка до commit викликає rollback.

За втрати відповіді після commit користувачеві потрібно звірити історію перед повтором. Ідемпотентний ключ повторного POST у поточній версії не реалізовано.

ER модель даних

Модель містить 9 сутностей. PK, FK і унікальність відповідають фактичним SQLAlchemy моделям та database/schema.sql. Обов’язковість і кардинальності пояснено на наступній сторінці.



Стрілка: таблиця з FK → батьківська таблиця. Номер відповідає зв’язку у словнику.

PK — первинний ключ; FK — зовнішній ключ; UQ — унікальність; NULL — необов’язкове поле.

Рисунок 4. ER схема з напрямком FK до батьківської таблиці.

Довідкова ціна товару і закупівельна ціна позиції надходження зберігаються окремо у цілих копійках. Історія містить залишок після операції для перевірки результату.

Словник сутностей та зв’язків

Сутність	Призначення та суттєві обмеження
users	Працівник; email UNIQUE; password_hash; role ∈ admin, keeper, manager; active.
auth_sessions	Серверний сеанс; SHA-256 token_hash PK; user_id NOT NULL; csrf; expires_at.
categories	Категорія; name і casefold name_key UNIQUE.
suppliers	Постачальник; назва і контакти. Не є зовнішньою інтеграцією.
products	SKU UNIQUE; обов’язкова категорія; optional постачальник; price_cents і min_stock ≥0.
stock	Один баланс на товар; product_id UNIQUE і NOT NULL; quantity ≥0.
receipts	Документ надходження; автор обов’язковий, постачальник optional; дата і note.
receipt_items	Позиція документа; товар і receipt обов’язкові; quantity >0; price_cents ≥0.
stock_movements	Рух; товар і автор обов’язкові; receipt optional; quantity >0; balance_after ≥0.

№	Зв’язок	Кардинальність
1	categories → products	1 категорія : 0..* товарів; товар має 1 категорію
2	suppliers → products	1 постачальник : 0..* товарів; товар має 0..1 постачальника
3	products → stock	1 товар : 0..1 рядок БД; застосунок створює рівно 1
4	users → auth_sessions	1 працівник : 0..* сеансів; сеанс має 1 працівника
5	suppliers → receipts	1 постачальник : 0..* документів; документ має 0..1 постачальника
6	users → receipts	1 працівник : 0..* документів; документ має 1 автора
7	receipts → receipt_items	1 документ : 0..* рядків БД; API вимагає 1–50 позицій
8	products → receipt_items	1 товар : 0..* позицій; позиція має 1 товар
9	products → stock_movements	1 товар : 0..* рухів; рух має 1 товар
10	users → stock_movements	1 працівник : 0..* рухів; рух має 1 автора
11	receipts → stock_movements	1 документ : 0..* рухів; рух має 0..1 документ

БД не гарантує існування дочірнього stock або хоча б однієї позиції документа; ці нижні межі забезпечує сервіс. Дублі товару в одному надходженні забороняє API, а не UNIQUE пара receipt_id/product_id.

Цілісність і трасування вимог

Розміщення бізнес правил

UI допомагає ввести дані, але не є джерелом довіри. API перевіряє формат, роль і стан товару; Stock Service визначає допустимість операції; БД виконує умовний UPDATE і перевіряє FK, UNIQUE та CHECK. Залишок, документ, позиції й аудит зберігаються одним commit.

Товари не видаляються разом з історією: адміністратор може архівувати товар лише за нульового залишку. Надходження збільшує кількість атомарним SQL increment; списання використовує WHERE quantity >= requested. За одночасних списань 15 і 15 зі stock 17 проходить лише одне, а кінцевий stock дорівнює 2.

Вимога	Компонент і дані	Екран	Доказ
US-01 / NFR-03	Auth; User, AuthSession	Вхід	auth, expiry, CSRF, logout тести
US-03 / NFR-01	Products; Product.name_search, sku	Товари	Unicode пошук, p95 тест
US-08 / NFR-02	Stock; Stock, Receipt, Item, Movement	Надходження	critical, multi-item, rollback
US-09 / NFR-02 / 05	Stock; Stock і Movement	Списання	negative, concurrent, React DOM
US-11 / NFR-06	Stock history; Movement і User	Історія	author, UTC, filters тести

Права та сеанси

Паролі — scrypt хеші; випадковий токен сеансу передається у HttpOnly/SameSite cookie, у БД зберігається його хеш. Окремий CSRF захищає записи. TTL сеансу — 8 годин; роль читається з БД під час запиту. Вихід або блокування відкликає доступ. У HTTPS розгортанні потрібно ввімкнути secure cookie.

Схема бази даних

Контракт API

Переходи прототипу та самоперевірка

Клікабельний прототип docs/Lab3/prototype.html містить вісім екранів: вхід, огляд, каталог, картка товару, надходження, списання, історія та користувачі. Він демонструє навігацію; реальні складські операції виконуються у React застосунку з GitHub.

№	Перехід або стан	Очікуваний результат
1	Вхід → огляд → товари	Авторизований користувач бачить дозволене меню
2	Пошук SSD-480 → картка	Товар знайдено, у свіжій демобазі stock 17
3	Картка → надходження 20	Success, новий stock 37
4	Історія після надходження	Є автор, час, кількість 20 і balance_after 37
5	Картка → списання 40	Error, stock 37, кількість і коментар збережені
6	Виправлення на 3 → проведення	Success, stock 34
7	Вихід → вхід менеджера	Доступне читання, запис через API заборонено

Фактичні результати

Я виконав технічну самоперевірку. На клоні GitHub пройдено 39 API тестів і production збірку. Раніше задокументований React DOM прогін із реальним API перевірив вхід, пошук, надходження, помилкове списання, історію та роль менеджера. У DOM сценарії товар списувався до нуля для перевірки архівації; сценарій 17 → 37 → 34 підтверджено API тестом.

Незалежного людського walkthrough не проведено, час пошуку ≤30 с не виміряно. Chromium/Firefox та візуальну перевірку на мобільному екрані не підтверджено. Я не заявляю ці перевірки завершеними. Інтерфейс потребує окремого локального прогону.

Зауваження і зміни

Виявлена проблема або ризик	Опрацювання
Success зникає під час фонового refresh	Збережено вже завантажений вміст; документовано DOM PASS
Надмірне списання може змінити stock	Умовний SQL decrement; 409 і незмінна історія; pytest PASS
Незрозуміло, що може менеджер	Матриця прав, обмеження UI і API; pytest та DOM PASS

Архітектурні рішення ADR

ADR 001 Технологічний стек

Статус: прийнято мною, єдиним автором проєкту. Контекст: невеликий навчальний вебзастосунок, короткий цикл розробки, окремі UI й API та локальна база без зовнішніх сервісів.

Рішення: React + Vite, Flask, Flask-SQLAlchemy/SQLAlchemy та SQLite; готовий SPA віддає Flask з того самого origin; локальний сервер — Waitress. Логічні модулі розміщені в одному backend застосунку.

Альтернативи: Django, Node.js backend або серверний рендеринг. Я не обрав їх у цій версії, щоб зберегти просту структуру і узгодженість із початковим стеком проєкту. Вони можливі для інших умов, але зміна стеку не покращує правило складського балансу сама по собі.

Наслідки: UI та API можна тестувати окремо; у розробці потрібен Vite proxy. SQLite вимагає постійного диска і серіалізує записи. Високе навантаження й міграції існуючої схеми потребують окремого проєктування.

ADR 002 Дані транзакції та API

Статус: прийнято мною. Контекст: після операції stock і журнал повинні відповідати одне одному; одночасні запити не повинні створити від’ємну кількість або втратити оновлення.

Рішення: JSON/HTTP API, SQLite як головне сховище, Stock Service з одним commit. Надходження використовує атомарний increment; списання — conditional decrement. Увімкнено foreign_keys, WAL і busy_timeout; дані захищені FK, UNIQUE та CHECK.

Альтернативи: JSON файл, MongoDB, PostgreSQL від початку, читання і перезапис stock у Python. Простий файл не дає необхідних реляційних обмежень; неумовний перезапис може втратити конкурентне оновлення. PostgreSQL доцільно оцінити за більших навантажень.

Наслідки: помилка запису аудиту відкатує операцію; дві надмірні конкурентні спроби не проходять разом. Історія не редагується через API. Невідомий результат POST через втрату мережевої відповіді потребує звірки з історією; автоматичної ідемпотентності немає.

ADR 001 у репозиторії

ADR 002 у репозиторії

Висновки і контрольні запитання

Я самостійно узгодив вимоги, компоненти, 9 сутностей і 8 екранів прототипу. Найбільше на проєктування вплинули цілісність stock, серверний доступ і аудит. Ці правила підтверджено тестами. GitHub і публічну дошку задач оформлено. Незалежний walkthrough, вимірювання часу пошуку та перевірка у двох браузерах ще потребують завершення.

Питання	Відповідь
Найсильніша вимога	NFR-02: Stock Service і транзакція об’єднують stock та аудит; це видно на компонентах і сценарії.
Захист даних	FK і UNIQUE захищають зв’язки; CHECK не допускає від’ємний stock; API і сервіс перевіряють додаткові правила.
Альтернативи	Django/Node.js для backend і PostgreSQL для БД; поточні умови дозволяють простий локальний стек.
Повідомлення про помилку	Error називає доступну кількість, зберігає поля і дає змогу виправити число та повторно підтвердити.
Де бізнес правило	UI підказує, API перевіряє доступ і формат, сервіс проводить операцію, БД гарантує атомарність і обмеження.
Новий сервіс або роль	Сервіс додається за межою системи з API адаптером; роль вимагає зміни матриці прав, CHECK і тестів.
Часткова відмова	До commit помилка аудиту веде до rollback. За втрати відповіді після commit потрібно звірити історію перед повтором.

У папці docs/Lab3/report-diagrams опубліковано уточнені контекстну, компонентну, ER та sequence діаграми. ER модель містить 9 сутностей, зокрема auth_sessions. Файли можна редагувати у PlantUML.

Використані матеріали

Методичні матеріали викладача «Проєктування архітектури даних та інтерфейсу»: розгорнутий текст на 20 сторінках і презентація на 55 сторінках.

Код і документація проєкту: https://github.com/chaek438/EasyStock

Перевірений commit 8e8a046cee04

Результати технічних перевірок